

数据结构题目

第一章绪论

1. 【2011.1】 设 n 是描述问题规模的非负整数，下面程序段的时间复杂度是(A)。

```
x=2;
while (x<n/2)
    x=x*2;
```

- A. $O(\log_2 n)$
- B. $O(n)$
- C. $O(n \log_2 n)$
- D. $O(n^2)$

A

2. 【2014.1】 下面程序段的时间复杂度是(C)。

```
count=0;
for(k=1; k<=n; k*=2)
    for(j=1; j<n; j++)
        count++;
```

- A. $O(\log_2 n)$
- B. $O(n)$
- C. $O(n \log_2 n)$
- D. $O(n^2)$

C

3. 算法分析的两个主要方面是(A)。

- A. 空间复杂度和时间复杂度
- B. 正确性和简单性
- C. 可读性和文档性
- D. 数据复杂性和程序复杂性

A

4. 以下说法正确的是(D)。

- A. 数据元素是数据的最小单位
- B. 数据项是数据的基本单位
- C. 数据结构是带有结构的各数据项的集合
- D. 一些表面上很不相同的数据可以有相同的逻辑结构

D

5. 数据结构是指(A)。

- A. 数据元素的组织形式
- B. 数据类型
- C. 数据存储结构
- D. 数据定义

A

6. 抽象数据类型的三个组成部分分别是()。

- A. 数据对象、数据关系和基本操作
- B. 数据元素、逻辑结构和存储结构

- C.数据项、数据元素和数据类型
- D.数据元素、数据结构和数据类型

A

7.数据在计算机存储器内表示时，物理地址与逻辑地址不相同的，称之为(C)。

- A.存储结构
- B.逻辑结构
- C.链式存储结构
- D.顺序存储结构

C

8.与数据元素本身的形式、内容、相对位置、个数无关的是数据的(C)。

- A.存储结构
- B.存储实现
- C.逻辑结构
- D.运算实现

C

9.在数据结构中，从逻辑上可以把数据结构分成(C)。

- A.动态结构和静态结构
- B.紧凑结构和非紧凑结构
- C.线性结构和非线性结构
- D.内部结构和外部结构

C

10.通常要求同一逻辑结构中的所有数据元素具有相同的特性，这意味着(B)。

- A.数据具有同一特点
- B.不仅数据元素所包含的数据项的个数要相同，而且对应数据项的类型要一致
- C.每个数据元素都一样
- D.数据元素所包含的数据项的个数要相等

B

二、基本概念与性质

- 1、数据，数据元素，数据项，数据对象，数据结构，逻辑结构，存储结构，抽象数据类型；
- 2、顺序表，链表，循环链表；
- 3、栈，队列，循环队列；
- 4、字符串相等、模式匹配；
- 5、地址计算、广义表的长度和深度。

三、算法设计题

- 1.【2009.42】已知一个带有表头结点的单链表，结点结构为（data,link），假设该链表只给出了头指针list。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第k个位置上的结点（k为正整数）。若查找成功，算法输出该结点data域的值，并返回1；否则，只返回0。要求：
 - 1) 描述算法的基本设计思想；
 - 2) 描述算法的详细实现步骤；
 - 3) 根据设计思想和实现步骤，采用C/C++/Java语言描述算法，关键之处给出简要注释。

2. 【2010.42】 设将 $n(n>1)$ 个整数存放于一维数组 R 中。试设计一个在时间和空间两方面都尽可能高效的算法，将 R 中保存的序列循环左移 $p(0<p<n)$ 个位置，即将 R 中的数据由 $(x_0, x_1, \dots, x_{n-1})$ 变换为 $(x_p, x_{p+1}, \dots, x_{n-1}, x_0, x_1, \dots, x_{p-1})$ 。要求：

- 1) 给出算法的基本设计思想；
- 2) 根据设计思想，采用 C/C++/Java 语言描述算法，关键之处给出简要注释；
- 3) 分析说明你所设计算法的时间复杂度和空间复杂度。

3. 假设以带头结点的循环链表表示队列，并且只设一个指针指向队尾元素结点（注意：不设头指针），试编写相应的置空队列、判断队列是否为空、入队和出队等算法。

4. 写一个算法，统计在输入字符串中各个不同字符出现的频度（字符串中的合法字符为 A~Z 这 26 个字母和 0~9 这 10 个数字）。

第二章 线性表

1. 线性表 $L=(a_1, a_2, \dots, a_n)$ ，下列说法正确的是()。

- A. 每个元素都有一个直接前驱和一个直接后继
 - B. 线性表中至少有一个元素
 - C. 表中诸元素的排列必须是由大到小或由小到大
 - D. 除第一个和最后一个元素外，其它每个元素都有一个且仅有一个直接前驱和直接后继
- D

2. 顺序表中第一个元素的存储地址是 100，每个元素的长度为 2，则第 5 个元素的地址是()

- 。
- A. 110

B.108

C.100

D.120

B

3.向一个有 127 个元素的顺序表中插入一个新元素并保持原来顺序不变, 平均要移动的元素个数为()。

A.8

B.63.5

C.63

D.7

B

4.在线性表的下列运算中, 不改变数据元素之间结构关系的运算是()。

A.插入

B.删除

C.排序

D.定位

D

5.线性表若采用链式存储结构时, 要求内存中可用存储单元的地址()。

A.必须是连续的

B.部分地址必须是连续的

C.一定是不连续的

D.连续或不连续都可以

D

6.在单链表中, 要将 s 所指结点插入到 p 所指结点之后, 其语句应为()。

A.s->next=p+1;p->next=s;

B.(*p).next=s;(*s).next=(*p).next;

C.s->next=p->next;p->next=s->next;

D.s->next=p->next;p->next=s;

D

7.对于一个头指针为 head 的带头结点的单链表, 判定该表为空表的条件是()。

A.head=NULL

B.head->next=NULL

C.head->next==head

D.head!=NULL

B

8.已知两个长度分别为 m 和 n 的升序链表, 若将它们合并成为一个长度为 m+n 的降序链表, 则最坏情况下的时间复杂度是()。

A.O(n)

B.O(m*n)

C.O(min(m,n))

D.O(max(m,n))

D

9.在循环链表中, 将头指针改为尾指针(rear)后, 其首元结点和尾结点的存储位置分别是()。

。

A.rear 和 rear->next->next

B.rear->next 和 rear

C.rear->next->next 和 rear

D.rear 和 rear->next

C

10.已知一个带有表头结点的双向循环链表 L，结点结构为 prev，data，next。其中，prev 和 next 分别指向其直接前驱和直接后继结点的指针。现要删除指针 p 所指的结点，正确的语句序列是()。

A.p->next->prev=p->prev;p->prev->next=p->prev;free(p);

B.p->next->prev=p->next;p->prev->next=p->next;free(p);

C.p->next->prev=p->next;p->prev->next=p->prev;free(p);

D.p->next->prev=p->prev;p->prev->next=p->next;free(p);

D

二、算法设计题

1.【2009.42】已知一个带有表头结点的单链表，结点结构为(data,link)，假设该链表只给出了头指针 list。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点(k 为正整数)。若查找成功，算法输出该结点 data 域的值，并返回 1；否则，只返回 0。要求：

- 1) 描述算法的基本设计思想；
- 2) 描述算法的详细实现步骤；
- 3) 根据设计思想和实现步骤，采用 C/C++/Java 语言描述算法，关键之处给出简要注释。

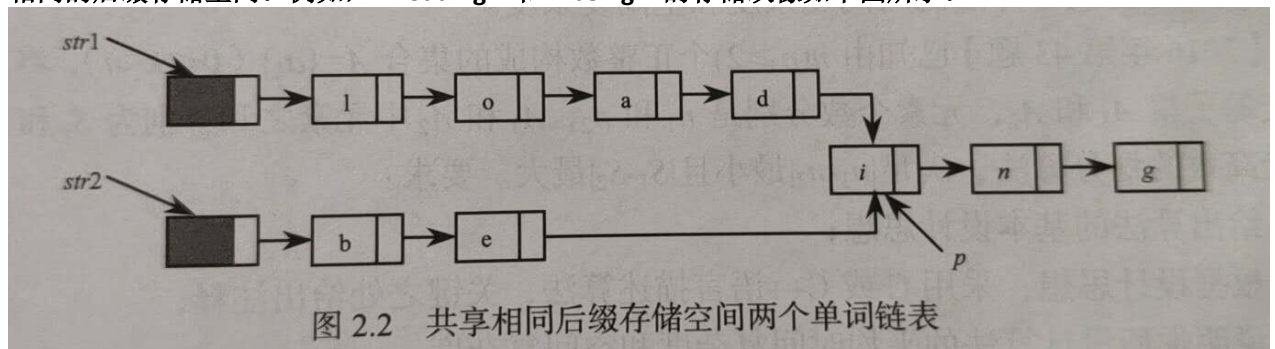
2.【2010.42】设将 $n(n>1)$ 个整数存放于一维数组 R 中。试设计一个在时间和空间两方面都尽可能高效的算法，将 R 中保存的序列循环左移 p($0<p<n$) 个位置，即将 R 中的数据由 $(x_0, x_1, \dots, x_{n-1})$ 变换为 $(x_p, x_{p+1}, \dots, x_{n-1}, x_0, x_1, \dots, x_{p-1})$ 。要求：

- 1) 给出算法的基本设计思想；
- 2) 根据设计思想，采用 C/C++/Java 语言描述算法，关键之处给出注释；
- 3) 说明你所设计算法的时间复杂度和空间复杂度。

3.【2011.42】一个长度为 L($L \geq 1$) 的升序序列 S，处在第 $\lfloor L/2 \rfloor$ 个位置的数称为 S 的中位数。例如，若序列 $S_1=(11,13,15,17,19)$ ，则 S_1 的中位数是 15。两个序列的中位数是含它们所有元素的升序序列的中位数。例如，若 $S_2=(2,4,6,8,20)$ ，则 S_1 和 S_2 的中位数是 11。现有两个等长升序序列 A 和 B，试设计一个在时间和空间两方面都尽可能高效的算法，找出两个序列 A 和 B 的中位数。要求：

1) 给出算法的基本设计思想；2) 根据设计思想，采用 C/C++/Java 语言描述算法，关键之处给出简要注释；3) 说明你所设计算法的时间复杂度和空间复杂度。

4. 【2012.42】假定用带头结点的单链表保存单词，当两个单词有相同的后缀时，则可共享相同的后缀存储空间。例如，“loading”和“being”的存储映像如下图所示。



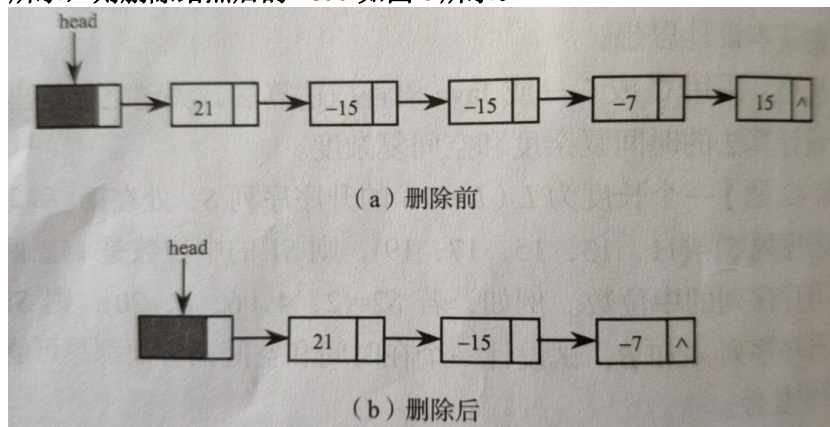
设 $str1$ 和 $str2$ 分别指向两个单词所在单链表的头结点，链表结点结构为 $(data, next)$ ，请设计一个时间上尽可能高效的算法，找出由 $str1$ 和 $str2$ 所指的链表共同后缀的起始位置(如图中字符 i 所在结点的位置 p)。要求：

- 1) 给出算法的基本设计思想；
- 2) 根据设计思想，采用 C/C++/Java 语言描述算法，关键之处给出简要注释；
- 3) 说明你所设计算法的时间复杂度。

5. 【2013.41】已知一个整数序列 $A=(a_0, a_1, \dots, a_{n-1})$ ，其中 $0 \leq a_i < n (0 \leq i < n)$ ，若存在 $a_{p_1} = a_{p_2} = \dots = a_{p_m} = x$ 且 $m > n/2 (0 \leq p_k < n, 1 \leq k \leq m)$ ，则称 x 为 A 的主元素。例如， $A=(0, 5, 5, 3, 5, 7, 5, 5)$ ，则 5 为主元素；又如 $A=(0, 5, 5, 3, 5, 1, 5, 7)$ ，则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中，请设计一个尽可能高效的算法，找出 A 中的主元素。若存在主元素，则输出该元素；否则输出 -1。要求：

- 1) 给出算法的基本设计思想；2) 根据设计思想，采用 C/C++/Java 语言描述算法，关键之处给出简要注释；
- 3) 说明你所设计算法的时间复杂度。

6. 【2015.41】用单链表保存 m 个整数，结点结构为 $(data, link)$ ，且 $|data| \leq n$ (n 为正整数)。现要求设计一个时间复杂度尽可能高效的算法，对于链表中 $data$ 的绝对值相等的结点，仅保留第一次出现的结点而删除其余绝对值相等的结点。例如，若给定的单链表 $head$ 如图 a 所示，则删除结点后的 $head$ 如图 b 所示。



要求：

- 1) 给出算法的基本设计思想；
- 2) 使用 C/C++ 语言，给出单链表结点的数据类型定义；
- 3) 根据设计思想，采用 C/C++ 语言描述算法，关键之处给出注释；
- 4) 说明你所设计算法的时间复杂度和空间复杂度。

7. 【2016.43】已知由 n ($n \geq 2$) 个正整数构成的集合 $A = \{a_k\} (0 \leq k < n)$ ，将其划分为两个不相交的子集 A_1 和 A_2 ，元素个数分别是 n_1 和 n_2 ， A_1 和 A_2 中元素之和分别为 S_1 和 S_2 。设计一个尽可能高效的划分算法，满足 $|n_1 - n_2|$ 最小且 $|S_1 - S_2|$ 最大。要求：

- 1) 给出算法的基本设计思想；
- 2) 根据设计思想，采用 C/C++ 语言描述算法，关键之处给出简要注释；
- 3) 说明你所设计算法的平均时间复杂度和空间复杂度。

`h2=new snode; q=h2;p=h1;`

`while(p)`

`if (p->next==NULL) p=NULL;`

`else`

`{q->next=p->next,q=q->next,p->next=q->next,p=p->next; }`

`q->next=NULL; q=h2; h2=h2->next; free q;`

以 h_1 为首指针的链表中，结点值依次为 $1, 2, 3, \dots, n$ (n 是某个具体的整数值)。试写出执行下面的程序段后，以 h_1 和 h_2 为首指针的链表中结点值的排列次序。

第三章 栈与队列

1. 【2010.1】若元素 a,b,c,d,e,f 依次进栈，允许进栈、出栈操作交替进行，但不允许连续三次进行出栈操作，则不可能得到的出栈序列是()。

A.d,c,e,b,f,a

B.c,b,d,a,e,f

C.b,c,a,e,f,d

D.a,f,e,d,c,b

D

2. 【2011.2】元素 a,b,c,d,e 依次进入初始为空的栈中，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素 d 开头的序列个数是()。

A.3

B.4

C.5

D.6

B

3. 【2014.2】假设栈初始为空将中缀表达式 $a/b+(c*d-e*f)/g$ 转换为等价的后缀表达式过程中，当扫描到 f 时，栈中的元素依次()。

A.+(*-

B.+(-*

C./+(*-*

D./+ -*

B

4. 【2009.1】为解决计算机主机与打印机速度不匹配问题，通常设一个打印数据缓冲区。主机将要输出的数据依次写入该缓冲区，打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应是()。

A.队列

B.栈

C.线性表

D.有序表

A

5. 【2009.2】设栈 S 和队列 Q 的初始状态均为空，元素 a,b,c,d,e,f,g 依次进栈 S。若每个元素出栈后立即进入队列 Q，且 7 个元素出队的顺序是 b,d,c,f,e,a,g,则栈 S 的容量至少是()。

A.1

B.2

C.3

D.4

C

6. 【2011.3】已知一个循环队列存储在一维数组 A[0..n-1]中，且队列非空时 front 和 rear 分别指向队头元素和队尾元素。若初始时队列为空，且要求第 1 个进入队列的元素存储在 A[0]处，则初始时 front 和 rear 的值分别是()。

A.0, 0

B.0, n-1

C.n-1, 0

D.n-1, n-1

B

7.若一个栈以向量 $V[1,...,n]$ 存储, 初始栈顶指针 top 设为 $n+1$, 则元素 x 进栈的正确操作是 ()。

A. $top++$; $V[top]=x$;

B. $V[top]=x$; $top++$;

C. $top--$; $V[top]=x$;

D. $V[top]=x$; $top--$;

C

8.最大容量为 n 的循环队列, 队尾指针是 $rear$, 队头是 $front$, 则队空的条件()。

A. $(rear+1)\%n==front$

B. $rear==front$

C. $rear+1==front$

D. $(rear-1)\%n==front$

B

9.一个递归算法必须包括()。

A.递归部分

B.终止条件和递归部分

C.迭代部分

D.终止条件和迭代部分

B

10.设有一个递归算法如下:

```
int x(int n)
```

```
{ if (n<=3) return 1;
```

```
    else return x(n-2)+x(n-4)+1;
```

```
}
```

则计算 $x(x(8))$ 时执行 x 函数的次数是()。

A.8

B.9

C.16

D.18

D

第四章 串

1.【2015.8】已知字符串 $s="abaabaabacacaa"$, 模式串 $t="abaabc"$ 。采用 KMP 算法进行匹配第一次出现“失配” ($s[i]!=t[j]$) 时, $i=j=5$, 则下次开始匹配时, i 和 j 的值分别是()。

A. $i=0, j=0$

B. $i=5, j=0$

C. $i=5, j=2$

D. $i=6, j=2$

C

2.串是一种特殊的线性表, 其特殊性体现在()。

A.可以顺序存储

B.数据元素是一个字符

- C.可以链式存储
- D.数据元素可以是多个字符

B

3.串的长度是指()。

- A.串中所含不同字母的个数
- B.串中所含字符的个数
- C.串中所含不同字符的个数
- D.串中所含非空格字符的个数

B

4.设主串的长度为 n ，子串的长度为 m ，那么 BF 算法的时间复杂度为()。

- A. $O(m)$
- B. $O(n)$
- C. $O(n \times m)$
- D. $O(n+m)$

C

5.设主串的长度为 n ，子串的长度为 m ，那么 KMP 算法的时间复杂度为()。

- A. $O(m)$
- B. $O(n)$
- C. $O(n \times m)$
- D. $O(n+m)$

D

6.设有两个串 S_1 和 S_2 ，求 S_2 在 S_1 中首次出现的位置的运算称作()。

- A.求子串
- B.判断是否相等
- C.模式匹配
- D.连接

C

7.串 "ababaaababaa" 的 next 数组值为()。

- A.012345678999
- B.0121211111212
- C.011234223456
- D.0123012322345

C

8.下面有关串的叙述中，错误的是()。

- A.串是字符的有限序列
- B.空串是由空格构成的串
- C.模式匹配是串的一种重要运算
- D.串既可以采用顺序存储，也可以采用链式存储

B

9.对于 KMP 算法，在模式匹配时指示主串匹配位置的指针()。

- A. 不会变大
- B. 不会变小
- C. 都有可能
- D. 无法判断

B

10.若串 $S = \text{"software"}$ ，其子串的数目是()。

A.8

B.37

C.36

D.9

B

第五章 数组和广义表

1.【2016.4】有一个 100 阶的三对角矩阵 M ，其元素 $m_{i,j}(1 \leq i \leq 100, 1 \leq j \leq 100)$ ，按行优先次序压缩存入下标从 0 开始的一维数组 N 中，元素 $m_{30,30}$ 在 N 中的下标是()。

A.86

B.87

C.88

D.89

B

2.设以行为主序存储二维数组 $A[1..100, 1..100]$ ，每个数据元素占 2 个存储单元，基地址为 10，则元素 $A[5,5]$ 的存储首地址是()。

A.808

B.818

C.1010

D.1020

B

3.设有数组 $A[1..8, 1..10]$ ，每个数据元素长度为 3 字节，数组首地址为 x ，当以列为主存放时，元素 $A[5,8]$ 的存储地址是()。

A. $x+141$

B. $x+180$

C. $x+222$

D. $x+225$

B

4.对特殊矩阵采用压缩存储的主要目的是()。

A.表达变得简单

B.对矩阵元素的存取变得简单

C.取得矩阵中多余的元素

D.减少不必要的存储空间

D

5.对 n 阶对称矩阵压缩存储时，需要顺序表的表长为()。

A. $n/2$

B. $n^2/2$

C. $n(n+1)/2$

D. $n(n-1)/2$

C

6.多维数组之所以有行优先顺序和列优先顺序两种存储方式是因为()。

- A.数组的元素处在行和列两个关系中
- B.数组的元素必须从左到右顺序排列
- C.数组的元素之间存在次序关系
- D.数组是多维结构，内存是一维结构

D

7.设广义表 $L = ((a,b,c))$ ，则 L 的长度和深度分别是()。

- A.1 和 1
- B.1 和 3
- C.1 和 2
- D.2 和 3

C

8.广义表 $L = (a, (b, c))$ ， $\text{tail}(L)$ 的结果是()。

- A.c
- B.b,c
- C.(b,c)
- D.((b,c))

D

9.下列说法正确的是()。

- 1) 稀疏矩阵压缩存储后，必会失去随机存取功能。
- 2) 若一个广义表的表头为空表，则此广义表亦为空表。
- 3) 广义表的取表尾运算，其结果通常是个表，但有时也可能是单元素值。
- 4) 从逻辑结构看， n 维数组是有多个 $n-1$ 维的数组构成。

- A.仅 1)，2)
- B.仅 1)，4)
- C.仅 2)，3)
- D.仅 3)，4)

B

10.下面说法不正确的是()。

- A.广义表的表头总是一个广义表
- B.广义表的表尾总是一个广义表
- C.广义表难以用顺序存储结构
- D.广义表可以是一个多层次的结构

A

第六章 树与二叉树

1.一个具有 1025 个结点的二叉树的高度 h 为()。

- A.11
- B.11 至 1025 之间
- C.10
- D.10 至 1024 之间

B

2.【2009.5】已知一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶子结点，则该完全二

叉树的结点个数最多是()。

A.39

B.52

C.111

D.119

C

3. 【2011.4】若一棵完全二叉树有 768 个结点，则该二叉树中叶子结点的个数为()。

A.257

B.258

C.384

D.385

C

4. 【2010.5】在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度为 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶子结点个数是()。

A.41

B.82

C.113

D.122

B

5. 【2011.5】若一棵二叉树的前序遍历序列和后序遍历序列分别为 1,2,3,4 和 4,3,2,1,则该二叉树的中序序列不会是()。

A.1,2,3,4

B.2,3,4,1

C.3,2,4,1

D.4,3,2,1

C

6. 【2012.3】若一棵二叉树的前序遍历序列为 a,e,b,d,c, 后序遍历序列为 b,c,d,e,a,则根结点的孩子结点为()。

A.只有 e

B.有 e,b

C.有 e,c

D.无法确定

A

7. 【2014.6】5 个字符有如下 4 种编码方案，不是前缀编码的是()。

A.01,0000,0001,001,1

B.011,000,001,010,1

C.000,001,010,011,100

D.0,100,110,1110,1100

D

8. 【2010.6】 $n(n \geq 2)$ 个权值均不相同的字符构成哈夫曼树，关于该树的叙述中，错误的是()。

A.该树一定是一棵完全二叉树

B.树中一定没有度为 1 的结点

C.树中两个权值最小的结点一定是兄弟

D. 树中任一非叶子结点的权值一定不小于下一层任一结点的权值

A

9. 【2014.5】将森林 F 转换成对应的二叉树 T, F 中叶子结点的个数等于()。

A. T 中叶子结点的个数

B. T 中度为 1 的结点个数

C. T 中左孩子指针为空的结点个数

D. T 中右孩子指针为空的结点个数

C

10. 【2016.5】若森林 F 有 15 条边、25 个结点, 则 F 包含树的个数是()。

A. 8

B. 9

C. 10

D. 11

C

11. 【2013.5】若 X 是后序线索二叉树中的叶子结点, X 存在左兄弟结点 Y, 则 X 的右线索指向()。

A. X 的父结点

B. 以 Y 为根的子树的最左下结点

C. X 的左兄弟结点 Y

D. 以 Y 为根的子树的最右下结点

A

12. 【2013.4】已知三叉树 T 中 6 个叶子节点的权值分别是 2、3、4、5、6、7, T 的带权(外部)路径长度最小是()。

A. 27

B. 46

C. 54

D. 56

B

13. 【2011.6】已知一颗有 2011 个结点的树, 其叶子结点个数为 116, 该树对应的二叉树中无右孩子的结点个数是()。

A. 115

B. 116

C. 1895

D. 1896

D

14. 设哈夫曼树中有 199 个结点, 则该哈夫曼树中的叶子结点数是()。

A. 99

B. 100

C. 101

D. 102

B

15. 一棵完全二叉树有 1001 个结点, 其中叶子结点的个数是()。

A. 250

B. 500

C.254

D.501

D

16.在下列存储形式中, ()不是树的存储形式。

A.双亲表示法

B.孩子链表表示法

C.孩子兄弟表示法

D.顺序存储表示法

D

17.由 3 个结点可以构造出()种不同的二叉树?

A.2

B.3

C.4

D.5

D

18.引入二叉线索树的目的是()。

A.加快查找结点的前驱或后继的速度

B.为了能在二叉树中方便地进行插入与删除

C.为了能方便的找到双亲

D.使二叉树的遍历结果唯一

A

19.一棵非空的二叉树的先序遍历序列与后序遍历序列正好相反, 则该二叉树一定满足()。

A.所有的结点均无左孩子

B.所有的结点均无右孩子

C.只有一个叶子结点

D.是一棵任意二叉树

C

20.【2015.3】下列选项给出的是从根分别到达两个叶子结点路径上的权值序列, 能属于同一棵哈夫曼树的是()。

A.24,10,5 和 24,10,7

B.24,10,5 和 24,12,7

C.24,10,10 和 24,14,11

D.24,10,5 和 24,14,6

D

二. 算法设计题

【2014.41】二叉树的带权路径长度(WPL)是二叉树中所有叶子结点的带权路径长度之和。给定一棵二叉树T, 采用二叉链表存储, 结点结构为(left, weight, right), 其中叶子结点的weight域保存该结点的非负权值。设root为指向根结点的指针, 请设计求T的WPL的算法。要求:

1) 给出算法的基本设计思想;

2) 使用c/c++语言, 给出二叉树结点的数据类型定义;

3) 根据设计思想, 采用c/c++语言描述算法, 关键之处给出注释。

算法思想:

利用递归的算法可以求解 WPL，已知：

WPL=树中所有叶子结点的带权路径长度之和=左子树中所有叶子结点的带权路径长度之和+右子树中所有叶子结点的带权路径长度之和。

叶子结点的带权路径长度=该结点的 weight 域的值×该结点的深度。

设根结点的深度为 0，若某结点的深度为 d 时，则其孩子结点的深度为 d+1。

算法的具体递归思路如下：

- 1) 如果是空树则递归结束，返回 0；
- 2) 如果是叶子结点则返回 weight×d；
- 3) 如果是其他情况，则返回左子树中所有叶子结点的带权路径长度之和+右子树中所有叶子结点的带权路径长度之和。

二叉树结点的数据类型定义如下：

```
typedef struct BiTNode
{
    int weight;
    struct BiTNode *left,*right;
} BiTNode,*BiTree;
```

算法描述：

```
int WPL(BiTree &T, int d)
{
    //求二叉树 T 的带权路径长度，调用时 T 指向二叉树根结点，d 为 0
    if (T==NULL)
        return 0;    //空树，递归结束，返回 0
    if (T->left==NULL && T->right==NULL)
        return (T->weight*d);    //叶子结点，返回 weight*d
    else    //否则递归调用
        return (WPL(T->left,d+1)+WPL(T->right,d+1));
}
```

第七章 图

一、选择题

1. 在一个图中，所有顶点的度数之和等于图的边数的（ ）倍。

A.1/2 B.1 C.2 D.4

C

2. 具有 n 个顶点的连通图用邻接矩阵表示时，该矩阵至少有（ ）个非零元素。

A.n B.2(n-1) C.n/2 D.n²

B

3. 【2010.7】若无向图 G=(V, E) 中含 7 个顶点，要保证图 G 在任何情况下都是连通的，则需要的边数最少是（ ）。

A.6 B.15 C.16 D.21

C

4. 若从无向图的任意一个顶点出发，进行一次深度优先搜索即可以访问图中所有的顶点，则该图一定是（ ）图。

A.非连通 B.连通 C.强连通 D.有向

B

5. 【2009.7】 下列关于无向连通图特性的叙述中，正确的是（ ）。

- I.所有顶点的度之和为偶数。
- II.边数大于顶点个数减 1。
- III.至少有一个顶点的度为 1。

- A.只有 I B.只有 II
- C.I 和 II D.I 和 III

A

6. 【2013.7】 设图的邻接矩阵如图所示，各顶点的度依次是（ ）。

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

- A.1, 2, 1, 2 B.2, 2, 1, 1
- C.3, 4, 2, 3 D.4, 4, 2, 2,

C

7. 下列关于最小生成树的说法中，正确的是（ ）。

- I.最小生成树的代价唯一。
- II.权值最小的边一定会出现在所有的最小生成树中。
- III.用 Prim 算法从不同顶点开始得到的最小生成树一定相同。
- IV.使用 Prim 算法和 Kruskal 算法得到的最小生成树总不相同。

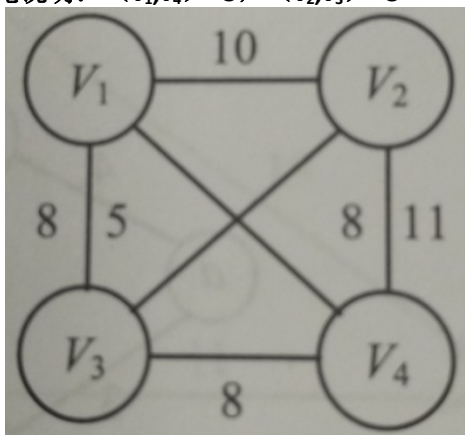
- A.只有 I B.只有 II C.I 和 III D.II 和 IV

A

8. 【2015.6】 求下面所示带权图的最小（代价）生成树时，可能是克鲁斯卡尔（Kruskal）算法第二次选中但不是普里姆（Prim）算法（从 V_4 开始）第二次选中的边是（ ）。

- A. (V_1, V_3) B. (V_1, V_4)
- C. (V_2, V_3) D. (V_3, V_4)

补充说明： $(v_1, v_4) = 5$, $(v_2, v_3) = 8$



C

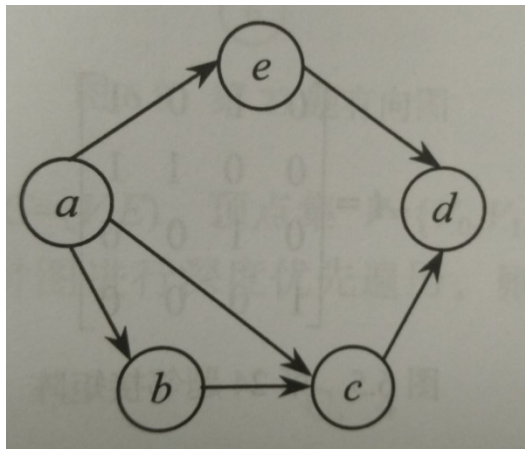
9. 【2012.6】 若用邻接矩阵存储有向图，矩阵中主对角线以下的元素均为零，则关于该图拓扑排序序列的结论是（ ）。

- A.存在，且唯一 B.存在，且不唯一
- C.存在，可能不唯一 D.无法确定是否存在

C

10. 【2010.8】对下面的有向图进行拓扑排序，可以得到不同的拓扑排序序列的个数是（）。

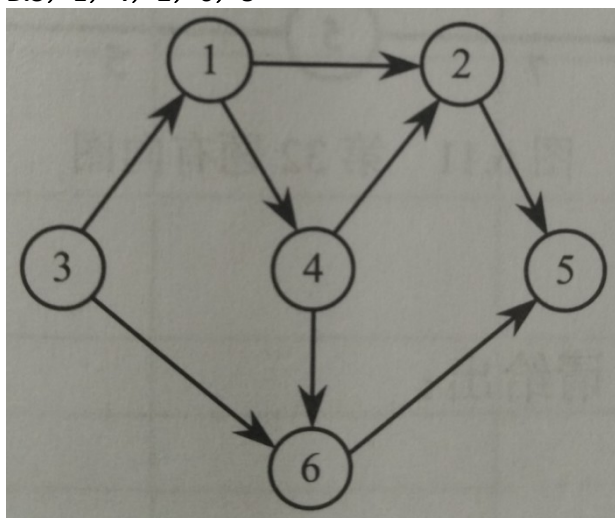
- A.4 B.3
C.2 D.1



B

11. 【2014.7】对下面的有向图进行拓扑排序，得到的拓扑序列可能是（）。

- A.3, 1, 2, 4, 5, 6
B.3, 1, 2, 4, 6, 5
C.3, 1, 4, 2, 5, 6
D.3, 1, 4, 2, 6, 5



D

12. G 是一个非连通无向图，共有 28 条边，则该图至少有（）个顶点。

- A.7 B.8 C.9 D.10

C

13. 图的 BFS 生成树的树高与 DFS 生成树的树高相比（）。

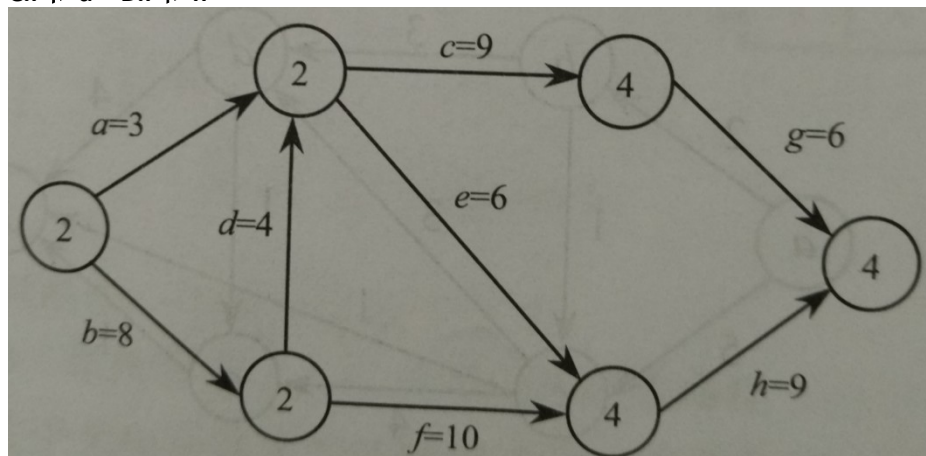
- A.小 B.相等
C.小或相等 D.大或相等

C

14. 【2013.9】下图所示 AOE 网表示一项包含 8 个活动的工程。通过同时加快若干活动的进度可以缩短整个工程的工期。下列选项中，加快其进度就可以缩短工程工期的是（）。

- A.c 和 e B.d 和 e

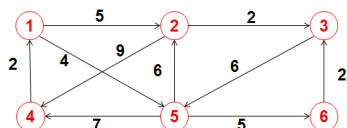
C.f 和 d D.f 和 h



C

二、填空题

- 1.具有 n 个顶点的有向图最多有 () 条边。
- 2.() 算法适合构造一个稠密图 G 的最小生成树。
- 3.[2012.5]对于有 n 个顶点、 e 条边且使用邻接表存储的有向图进行广度优先遍历，其算法的时间复杂度是 ()。
- 4.[2015.5]设有向图 $G=(V, E)$ ，顶点集 $V=\{V_0, V_1, V_2, V_3\}$ $E=\{< V_0, V_1 >, < V_0, V_2 >, < V_0, V_3 >, < V_1, V_3 >\}$ ，若从顶点 V_0 开始对图进行深度优先遍历，则可能出现的不同遍历序列有 () 个。
- 5.[2016.7]使用 Dijkstra 算法求图中从顶点 1 到其他各顶点的最短路径，依次得到的各最短路径的目标顶点是 ()。



1、 $n(n-1)$ 2、Prim 3、 $O(n+e)$ 4、5 5、5, 2, 3, 6, 4

三、判断题

- 1.用邻接矩阵存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中的顶点个数有关，而与图的边数无关。 ()
- 2.强连通分量是有向图中的极大强连通子图。 ()
- 3.图的广度优先搜索算法通常采用非递归算法求解。 ()
- 4.采用不同的遍历方法，所得到的无向图的生成树总是相同的。 ()
- 5.拓扑排序是指结点的值是有序排序的。 ()

1、对 2、对 3、对 4、错 5、错

四、应用题

1.[2011.41]已知有 6 个顶点（编号为 0~5）的有向带权图 G，其邻接矩阵 A 为上三角矩阵，按行为主序（行优先）保存在如下的一维数组中。

4	6	∞	∞	∞	5	∞	∞	∞	4	3	∞	∞	3	3
---	---	----------	----------	----------	---	----------	----------	----------	---	---	----------	----------	---	---

要求：

- 1) 写出图 G 的邻接矩阵 A；
- 2) 画出有向带权图 G；
- 3) 求图 G 的关键路径，并计算该关键路径的长度

2.[2009.41]带权图（权值非负，表示边连接的两个顶点间的距离）的最短路径问题是找出从初始顶点到目标顶点之间的一条最短路径。假定从初始顶点到目标顶点之间存在路径，现有一种解决该问题的方法：

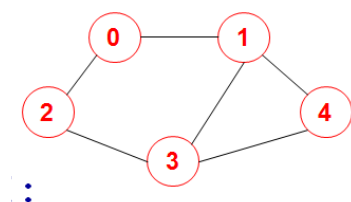
- 1) 设最短路径初始时仅包含初始顶点，令当前顶点 u 为初始顶点；
- 2) 选择离 u 最近且尚未在最短路径中的一个顶点 v，加入到最短路径中，修改当前顶点 $u=v$ ；
- 3) 重复步骤 2，直到 u 是目标顶点时为止。

请问上述方法能否求得最短路径？若该方法可行，请证明之；否则，请举例说明。

3.[2015.42]已知有 5 个顶点的图 G 如下

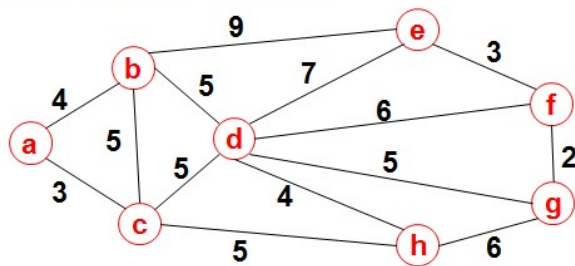
请回答下列问题：

- 1) 写出图 G 的邻接矩阵 A（行、列下标从 0 开始）。
- 2) 求 A^2 ，矩阵 A^2 中位于 0 行 3 列元素值的含义是什么？
- 3) 若已知具有 n（ $n \geq 2$ ）个顶点的邻接矩阵为 B，则 B^m （ $2 \leq m \leq n$ ）非零元素的含义是什么？

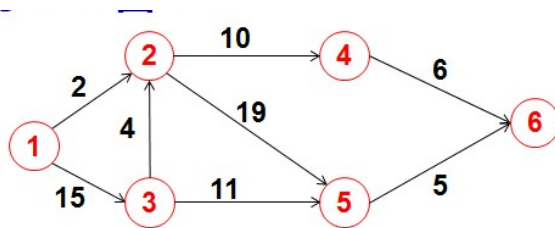


4. 已知无向图，请给出：

- 1) 邻接矩阵；
- 2) 邻接表；
- 3) 最小生成树。



5. 试对 AOE-图：



- 1) 求这个工程最早可能在什么时间结束；
- 2) 求每个活动的最早开始时间和最迟开始时间；
- 3) 确定哪些活动是关键活动。

五、算法设计题

1. 分别以邻接矩阵作为存储结构，实现以下图的基本操作：

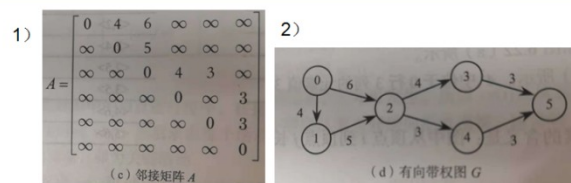
- 1) 增加一个新顶点 v , $\text{InsertVex}(G, v)$;
- 2) 删除顶点 v 及其相关的边, $\text{DeleteVex}(G, v)$;
- 3) 增加一条边 $\langle v, w \rangle$, $\text{InsertArc}(G, v, w)$;
- 4) 删除一条边 $\langle v, w \rangle$, $\text{DeleteArc}(G, v, w)$ 。

2. 一个连通图采用邻接表作为存储结构，设计一个算法，实现从顶点 v 出发的深度优先遍历的非递归过程。
3. 设计一个算法，求图 G 中距离顶点 v 的最短路径长度最大的一个顶点，设 v 可达其余各个顶点。
4. 试基于图的深度优先搜索策略写一个算法，判断以邻接表方式存储的有向图中是否存在由顶点 v_i 到顶点 v_j 的路径 ($i \neq j$)。
5. 采用邻接表存储结构，编写一个算法，判断无向图中任意给定的两个顶点之间是否存在一条长度为 k 的简单路径。

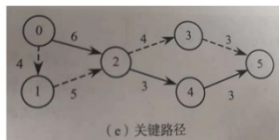
答案:

四、应用题

1、

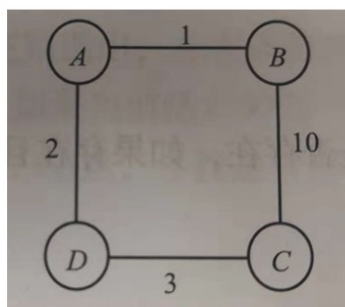


- 3) 关键路径 0-1-2-3-5, 关键活动 1,2,3,5, 关键路径长度 $4+5+4+3=16$



2、

答：该方法求得的路径不一定是最短路径。例如，对于如图所示的带权图，如果按照题中的求解原则，从A到C的最短路径为A->B->C，事实上最短路径为A->D->C。



3、

- 1) 图G的邻接矩阵A如图 (a)
- 2) 矩阵 A^2 如图 (b)， A^2 中位于0行3列的元素值3表示从顶点0到顶点3之间的长度为2的路径共有3条。
- 3) B^m ($2 \leq m \leq n$) 非零元素的含义是：图中从顶点i到顶点j长度为m的路径条数。

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix} \quad A^2 = \begin{bmatrix} 3 & 1 & 0 & 3 & 1 \\ 1 & 3 & 2 & 1 & 2 \\ 0 & 2 & 2 & 0 & 2 \\ 3 & 1 & 0 & 3 & 1 \\ 1 & 2 & 2 & 1 & 3 \end{bmatrix}$$

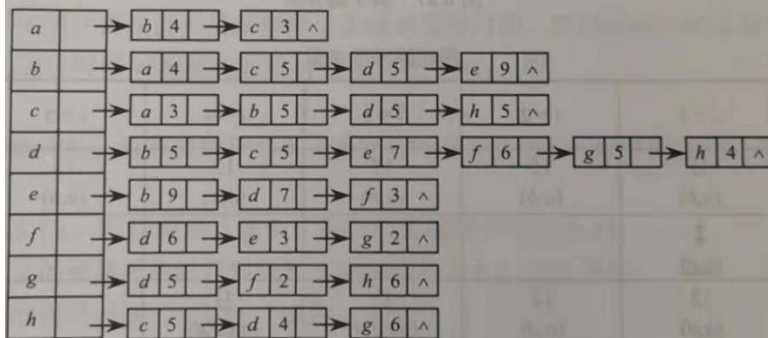
(a) 矩阵A (b) 矩阵 A^2

4、

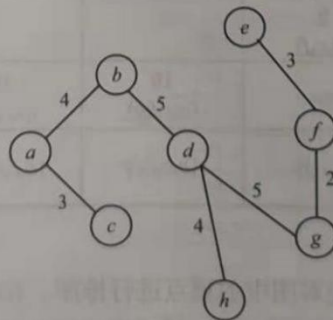
- 1) 邻接矩阵如图 (a)；
- 2) 邻接表如图 (b)；
- 3) 最小生成树如图 (c)

∞	4	3	∞	∞	∞	∞	∞
4	∞	5	5	9	∞	∞	∞
3	5	∞	5	∞	∞	∞	5
∞	5	5	∞	7	6	5	4
∞	9	∞	7	∞	3	∞	∞
∞	∞	∞	6	3	∞	2	∞
∞	∞	∞	5	∞	2	∞	6
∞	∞	5	4	∞	∞	6	∞

(a) 邻接矩阵



(b) 邻接表



(c) 最小生成树

5、

- 1) 如图 (a) 所示, 这个工程最早可能结束的时间为43;
- 2) 每个活动的最早开始时间和最迟开始时间如图 (b) ;
- 3) 关键活动包括<1,3>、<3,2>、<2,5>、<5,6>。

(a) 顶点事件的发生时间		
顶点 i	$ve(i)$	$vl(i)$
1	0	0
2	19	19
3	15	15
4	29	37
5	38	38
6	43	43

(b) 活动的开始时间			
活动 a_i	$e(i)$	$l(i)$	$l(i)-e(i)$
<1,2>	0	17	17
<1,3>	0	0	0
<3,2>	15	15	0
<2,4>	19	27	8
<2,5>	19	19	0
<3,5>	15	27	12
<4,6>	29	37	8
<5,6>	38	38	0

五、算法设计题

1、1) 算法描述

```
status InsertVex(AMGraph &G,int v)
{
    //在以邻接矩阵形式存储的无向图 G 上插入顶点 v
    if((G.vexnum+1)>MVNUM)
        return INFEASIBLE;           //判断插入操作是否合法
    G.vexnum++;                       //增加图的顶点数量
    G.vex[G.vexnum]=v;                //将新顶点 v 存入顶点表
    for(k=0;k<G.vexnum;k++)           //邻接矩阵相应位置的元素置为 1
        G.arcs[G.vexnum][k]=G.arcs[k][G.vexnum]=0;
    return OK;
}
```

2) 算法描述

```
status DeleteVex(AMGraph &G,int v)
{
    //以邻接矩阵形式存储的无向图 G 上删除顶点 v
    n=G.vexnum;
    if((m=LocateVex(G,v))<0)          //判断删除操作是否合法
        return ERROR;
    G.vexs[m]<->G.vexs[n];             //将待删除顶点交换到最后一个顶点
    for(i=0;i<n;i++)                  //将边关系随之交换
        {G.arcs[m][i]<->G.arcs[n][i];
         G.arcs[i][m]<->G.arcs[i][n];
        }
    G.vexnum--;                       //顶点总数减一
    return OK;
}
```

3) 算法描述

```
status InsertArc(AMGraph &G,int v,int w)
{
    //以邻接矩阵形式存储的无向图 G 上插入边 (v,w)
    if((i=LocateVex(G,v))<0)          //判断插入位置操作是否合法
        return ERROR;
    if((j=LocateVex(G,w))<0)
        return ERROR;
    if(i==j)
        return ERROR;
    G.arcs[i][j]=G.arcs[j][i];         //邻接矩阵上增加相应的边
    G.arcnum++;                         //边数加一
    return OK;
}
```

4) 算法描述

```
status DeleteArc(AMGraph &G,int v,int w)
{
    //以邻接矩阵形式存储的无向图 G 上删除边 (v,w)
    if((i=LocateVex(G,v))<0)          //判断插入位置操作是否合法
        return ERROR;
```

```

        if((j=LocateVex(G,w))<0)
            return ERROR;
        G.arcs[i][j]=G.arcs[j][i]=0;           //邻接矩阵上删除边
        G.arcnum--;                           //边数减一
        return OK;
    }

```

2.算法描述

```

void DFS(AMGraph &G,int v)
{
    //以第 v 个顶点出发非递归实现深度优先遍历图 G
    InitStack(&S);           //构造一个空栈
    push(S,v);               //顶点 v 入栈
    while(!StackEmpty(S))    //栈非空
    {
        pop(S,k);            //栈顶元素 k 出栈
        if(!visited[k])      //k 未被访问
        {
            print(k);         //访问第 k 个顶点
            visited[k]=true;
            p=G.vertices[k].firstarc; //p 指向 k 的边链表的第一个边结点
            while(p!=null)     //边结点非空
            {
                w=p->adjvex;    //表示 w 是 k 的邻接点
                if(!visited[w]) push(S,w); //如果 w 未访问, w 进栈
                p=p->nextarc;    //p 指向下一个边结点
            }
        }
    }
}

```

3.算法描述

```

int ShortestPathMAX(AMGraph G,int v0)
{
    //用 Dijkstra 算法求图 G 中距离顶点 v0 的最短路径长度最大的一个顶点 m
    n=G.vexnum;           //n 为 G 中顶点的个数
    for(v=0;v<n;++v)      //n 个顶点依次初始化
    {
        S[v]=false;
        D[v]=G.arcs[v0][v]; //将 v0 到各个顶点的最短路径长度初始化
        if(D[v]=0) D(v)=MaxInt;
        if(D[v]<MaxInt) Path[v]=v0;
        else Path[v]=-1;    //如果有弧, v 前驱置为 v0 否则为-1
    }
    //for
    S[v0]=ture;           //将 v0 加入 S
    D[v0]=0;              //源点到源点的距离为 0
    //初始化结束, 开始主循环, 每次求得 v0 到某个顶点 v 的最短路径, 将 v 加入 S 集
    for(i=1;i<n;++i) //对其余 n-1 个顶点, 依次进行计算
    {
        min=MaxInt;
        for(w=0;w<n;++w)
            if(!S[w]&&D[w]<min)

```

```

        {v=w;min=D[w];} //选择一条当前的最短路径，终点为 v
    S[v]=true;           //将 v 加入 S
    for(w=0;w<n;++w)     //更新从 v0 出发到集合 V-S 上所有顶点的最短路径长度
        if(!S[w]&&(D[v]+G.arcs[v][w]<D[w]))
            { D[w]=D[v]+G.arcs[v][w];           //更新 D[w]
              Path[w]=v;                         //更改 w 的前驱为 v
            }//if
    }//for
//最短路径求解完毕，设距离顶点 v0 的最短路径长度最大的一个顶点为 m
Max=D[0];
m=0;
for(i=1;i<n;i++)
    if(Max<D[i])    m=i;
return m;           //返回顶点下标
}

```

4.算法描述

```

int visited[MVNum];           //访问标志数组，其初始值为 false
int level=1;                  //递归进行的层数
int PathDFS(ALGraph G,int i,int j)
{
    //基于深度优先搜索判断有向图 G 中顶点 i 到 j 是否有路径，是返回 1，否返回 0
    if(i==j)    return 1;      //递归结束条件，首尾相遇，存在路径
    else
        {
            visited[i]=ture;    //顶点 i 访问标志为 ture
            for(p=G.vertices[i].firstarc;p;p=p->nextarc,level--)
                {
                    level++;    //递归层次增 1
                    k=p->adjvex;
                    if(!visited[k]&&PathDFS(G,k,j))
                        return 1;    //i 下游的顶点到 j 有路径
                }//for
        }//else
    if(level==1)    return 0;    //i 到 j 没有路径
}

```

5.算法描述

```

int visited[MAXSIZE];
int PathLenK(ALGraph G,int i,int j,int k)
{
    //判断邻接表方式存储的有向图 G 的顶点 i 到 j 是否存在长度为 k 的简单路径
    if(i==j&&k==0)    return 1;    //找到一条路径
    else if(k>0)
        {
            visited[i]=ture;
            for(p=G.vertices[i].firstarc;p;p=p->nextarc)
                {
                    //从结点 i 开始遍历，p 为 i 的边链表的第一个边结点
                    v=p->adjvex;
                    if(!visited[v])    //v 未被访问过
                        return 1;    //递归继续判断 i 到 j，且剩余路径长度减 1
                }
        }
    return 0;
}

```

```

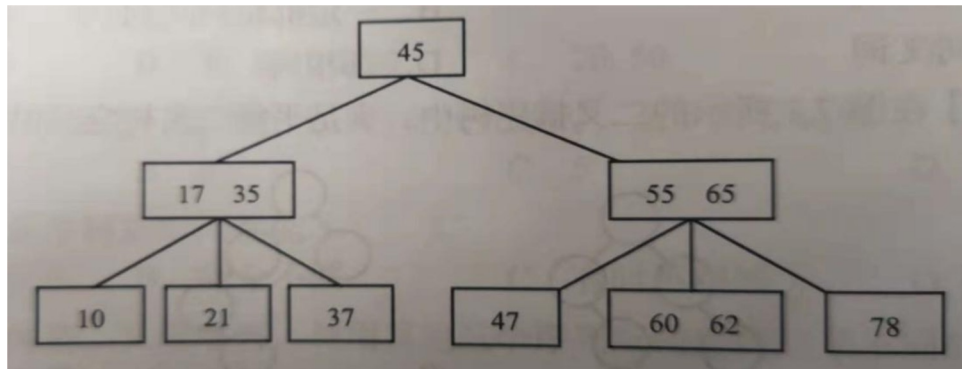
        }//for
        visited[i]=false;           //允许曾经被访问过的结点出现在另一条路径
中
        }//else
        return 0;           //没找到
    }

```

第九章 查找

一、选择题

- 适用于折半查找的表的存储方式及元素排列要求为（ ）。
 A. 链接方式存储，元素无序
 B. 链接方式存储，元素有序
 C. 顺序方式存储，元素无序
 D. 顺序方式存储，元素有序
D
- 在有序表{4,6,10,12,20,30,50,70,88,100}中进行折半查找。若查找表中元素 58，则依次与表中的（ ）比较大小，查找结果是失败。
 A. 20,70,30,50 B. 30,88,70,50
 C. 20,50 D. 30,88,50
A
- 对 22 个记录的有序表进行折半查找，当查找失败时，至少要比（ ）次关键字。
 A. 3 B. 4 C. 5 D. 6
B
- 【2010.9】已知一个长度为 16 的顺序表 L，其元素按关键字有序排列，若采用折半查找法查找一个不存在的元素，则比较的次数最多的是（ ）。
 A. 4 B. 5 C. 6 D. 7
B
- 【2011.7】对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是（ ）。
 A. 95,22,91,24,94,71 B. 92,20,9,34,88,35
 C. 21,89,77,29,36,38 D. 12,25,71,68,33,34
A
- 【2012.9】有一颗 3 阶 B-树，如图，删除关键字 78 得到一颗新 B-树，其最右叶子结点中所含的关键字是（ ）
 A. 60 B. 60,62 C. 62,65 D. 65
D



7.[2014.8]用哈希(散列)方法处理冲突(碰撞)时可能出现聚集(堆积)现象, 下列选项中, 会受到堆积现象直接影响的是 ()

- A. 存储效率
- B. 散列函数
- C. 装填(装载)因子
- D. 平均查找长度

D

8.[2014.9]在一颗具有 15 个关键字的 4 阶 B-树中, 含关键字的结点个数最多是 ()

- A. 5
- B. 6
- C. 10
- D. 15

D

9.[2010.9]已知一个长度为 16 的顺序表 L, 其元素按关键字有序排列, 若采用折半查找法查找一个不存在的元素, 则比较次数最多的是 ()

- A. 4
- B. 5
- C. 6
- D. 7

B

10.[2016.10]B+树不同于 B-树的特点之一是 ()

- A. 能支持顺序查找
- B. 结点中含有关键字
- C. 根结点至少有两个分支
- D. 所有叶子结点都在同一层上

A

二、应用题

1.对有序表{3,4,5,7,24,30,42,54,63,72,87,95}进行折半查找, 试回答下列问题:

- 1) 画出描述折半查找过程的判定树。
- 2) 若查找元素 54, 需依次与哪些元素比较?
- 3) 若查找元素 90, 需依次与哪些元素比较?
- 4) 假定每个元素的查找概率相等, 求查找成功时的平均查找长度。

2.在一棵空的二叉排序树中依次插入关键字序列为{12,7,17,11,16,2,13,9,21,4}, 请画出所得到的二叉排序树。

3. 设散列表的地址范围为 0~17，散列函数为： $H(\text{key}) = \text{key} \% 16$ ，用线性探测法处理冲突，输入关键字序列为

(10,24,32,17,31,30,46,47,40,63,49)，试回答下列问题：1) 画出散列表的示意图。

2) 若查找关键字 63，需依次与哪些关键字比较？

3) 若查找关键字 60，需依次与哪些关键字比较？

4) 假定每个关键字的查找概率相等，求查找成功时的平均查找长度。

4.[2010.41] 将关键字序列{7,8,30,11,18,9,14}散列存储到散列表中，散列表的存储空间是一个下标从 0 开始的一维数组散列，函数为： $H(\text{key}) = (\text{key} \times 3) \text{ MOD } T$ ，处理冲突用线性探测再散列法，要求装载因子为 0.7。问题：

1) 请画出所构造的散列表。

2) 分别计算等概率情况下，查找成功和查找不成功的平均查找长度。

5.[2013.42] 设包含 4 个数据元素的集合 $S = \{ \text{"do"}, \text{"for"}, \text{"repeat"}, \text{"while"} \}$ ，各元素的查找概率依次为： $p_1=0.35, p_2=0.15, p_3=0.15, p_4=0.35$ 。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

1) 若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

2) 若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

三、算法设计题

1. 试写出折半查找的递归算法。

2. 已知二叉排序树采用二叉链表存储结构，根节点的指针为 T，链结点的结构为 (lchild, data, rchild)，其中 lchild、rchild 分别指向该结点的左、右孩子的指针，data 域存放结点的数据信息。请写出递归算法，从小到大输出二叉排序树中所有数据值 $\geq x$ 的结点的数据。要求先找到第一个满足条件的结点后，再依次输出其他满足条件的结点。

3. 假设一棵平衡二叉树的每个结点都表明了平衡因子 b，试设计一个算法，求平衡二叉树的高度。

4. 分别写出在散列表中插入和删除关键字为 K 的一个记录的算法，设散列函数为 H，解决冲突的方法为链地址法。

5. 已知二叉树 T 的结点形式为 $(llink, data, count, rlink)$, 在树中查找值为 x 的结点, 若找到, 则计数($count$)加 1; 否则, 作为一个新结点插入到树中, 插入后仍为二叉排序树, 写出其非递归算法。

答案

二、应用题

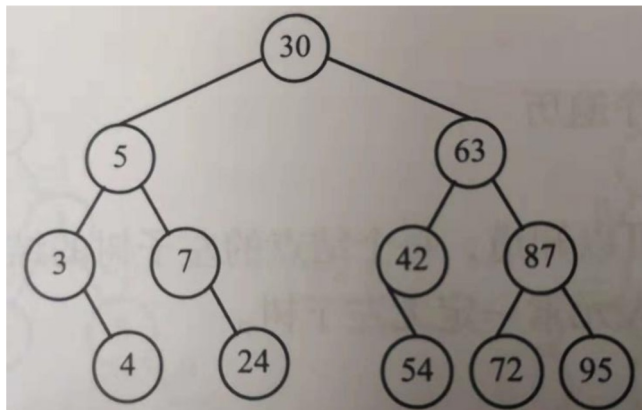
1) 判定树如图所示。

2) 查找元素 54, 需依次与 30、63、42、54 元素比较。

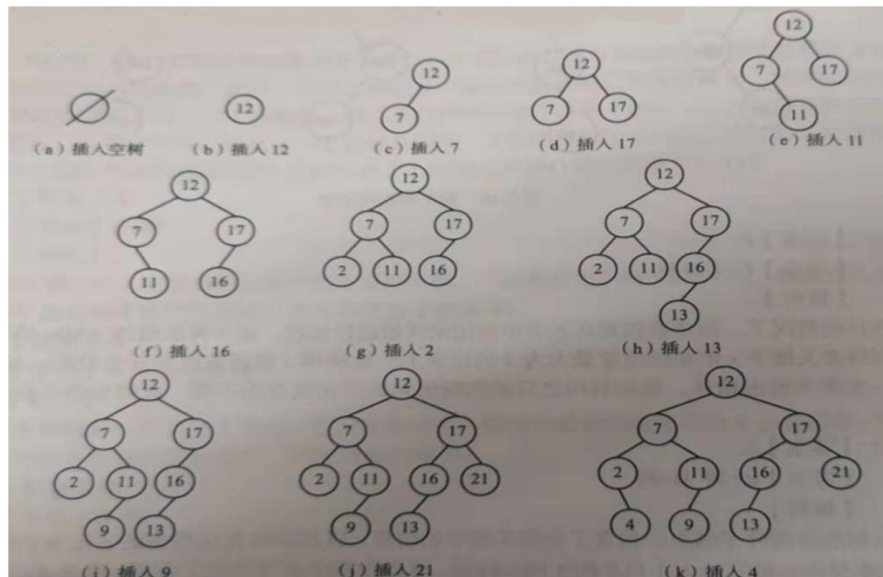
3) 查找元素 90, 需依次与 30、63、87、95 元素比较。

4) 查找时元素的比较次数总计为: $1+2*2+4*3+5*4=37$, 所以

$$ASL=37/12 \approx 3.08$$



2、



3、

1) 散列表如图

散列地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
关键字	32	17	63	49					24	40	10				30	31	46	47
比较次数	1	1	6	3					1	2	1				1	1	3	3

- 2) 查找63时, 首先与 $H(63)=63\%16=15$ 号单元的内容比较, 即63与31比较, 不匹配, 然后顺移, 与46、47、32、17、63相比, 一共比较了6次。
 3) 查找60时, 首先与 $H(60)=60\%16=12$ 号单元的内容比较, 因12号单元为空, 所以只比一次即可, 查找失败。
 4) 假定每个关键字的查找概率相等, 查找成功时的平均查找长度为:
 $ASL = (1+1+6+3+1+2+1+1+1+3+3) / 11 = 23/11$

4、

- 1) 因为装填因子是0.7, 关键字总个数为7, 所以存储空间长度为 $L=7/0.7=10$, 构造函数为: $H(key) = (key \times 3) \text{ MOD } 7$, 线性探测再散列函数为: $H(key) = (H(key) + d_i) \text{ MOD } 10$, ($d_i = 1, 2, 3, \dots, 9$)。构造的散列表如图:

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字	7	14		8		11	30	18	9		
比较次数	1	2		1		1	1	3	3		

- 2) 由上表可得, 等概率下查找成功时和查找失败时的平均查找长度为:
 $ASL_{succ} = (1+2+1+1+1+3+3) / 7 = 12/7$
 $ASL_{unsucc} = (3+2+1+2+1+5+4) / 7 = 18/7$

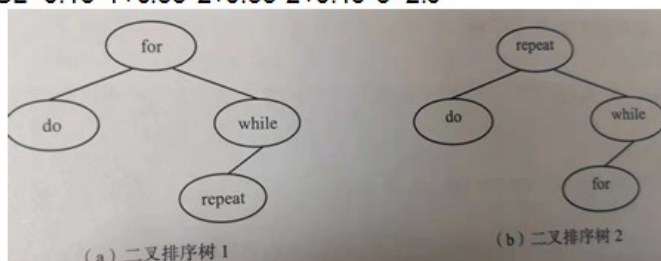
5、

【答案一】

- 1) 采用顺序存储结构, 数据元素按其查找概率降序排列。
 采用顺序查找方法, 查找成功时的平均查找长度为:
 $ASL = 0.35 \times 1 + 0.35 \times 2 + 0.15 \times 3 + 0.15 \times 4 = 2.1$
 2) 采用链式存储结构, 数据元素按其查找概率降序排列, 构成单链表。
 采用顺序查找方法, 查找成功时的平均查找长度为:
 $ASL = 0.35 \times 1 + 0.35 \times 2 + 0.15 \times 3 + 0.15 \times 4 = 2.1$

【答案二】

- 采用二叉链表存储结构, 构造二叉排序树, 可得到如图所示的两颗不同形态的二叉排序树。采用二叉排序树的查找方法, 查找成功时的平均查找长度为:
 $ASL = 0.15 \times 1 + 0.35 \times 2 + 0.35 \times 2 + 0.15 \times 3 = 2.0$



三、算法设计题

1.算法描述

```
int BinSearch_Cur(SSTable ST,KeyType key,int low,int high)
{
    //在有序表 ST 中折半查找其关键字为 key 的数据元素，成功返回 k 的位置，失败返回 0
    if(low>high) return 0;          //查找不到返回 0
    if(low<high)
    {
        mid=(low+high)/2;          //mid 取值为中间值
        if(ST.R[mid]==key)
            return mid;
        else if(key<ST.R[mid])
            return BinSearch_Cur(ST,key,low,mid-1); //对左子表递归查找
        else
            return BinSearch_Cur(ST,key,mid+1,high); //对右子表递归查找
    }
}
```

2.算法描述

```
Void Print(BSTree T)
{
    //中序输出以 T 为根的二叉排序树的所有结点
    if(T)
    {
        Print(T->lchild);
        print(T->data);
        Print(T->rchild);
    }
}

Void PrintAllx(BSTree t,ElemType x)
{
    //在二叉排序树 T 中，查找值>=x 的结点并输出
    p=T;
    if(p)
    {
        while(p&&p->data<x) //沿右分支向下，找到第一个值>=x 的结点
            p=p->rchild; T=p; //T 指向结点是值>=x 的结点的树的根
        if(p) //找到第一个值<x 的结点
        {
            f=p; p=p->lchild;
            while(p&&p->data<x) //沿左分支向下，找到第一个值<x 的结点
            {
                f=p; p=p->lchild; //f 是 p 的双亲结点的指针，并指向第一个值>=x 的结点
            }
            if(p) f->lchild=null; //双亲与找到的第一个值<x 的结点断开
            Print(T); //输出以 T 为根的子树
        }
    }
}
```

3.算法描述

```
Int Height(BSTree T)
{
    //求平衡二叉树 T 的高度
    level=0; //记录树的高度
}
```

```

    p=T;
    while(p)
    {level++;          //树的高度增 1
      if(p->b<0)
        p=p->rchild;   //b=-1 沿右分支向下
      else
        p=p->lchild;   //b>=0 沿左分支向下
    }
    return level;      //返回二叉平衡树的高度
}

```

4.算法描述

```

Bool Insert_K()
{scan(&data);
  ant=H(data);
  p=HT[ant]; //找到该散列地址代表的位置
  while(p->next) //确定插入位置
    { if(p->next->data==data)
      {return false;
        p=p->next;
      }
    }
  s=newLNode;
  s->data=data;
  s->next=p->next;
  p->next=s; //插入该结点置于链表尾部
  return true;
}

```

```

Bool Delete_K()
{scan(&data);
  ant=H(data);
  p=HT[ant]; //找到该散列地址代表的位置
  while(p->next) //确定删除位置
    { if(p->next->data==data)
      {s=p->next;
        p->next=s->next;
        delete s; //删除该结点
        return true;
      }
      p=p->next; //遍历下一个结点
    }
  return false;
}

```

5.算法描述

```

Void SearchBST(BSTree &T, int x)
{s=new BiTnode;

```


{5,8,12,19,28,20,15,22}是小根堆，插入关键字3，调整后得到的小根堆是（）。

A.3,5,12,8,28,20,15,22,19

B.3,5,12,19,20,15,22,8,28

C.3,8,12,5,20,15,22,28,19

D.3,12,5,8,28,20,15,22,19

7. 【2010.11】对一组数据{2,12,16,88,5,10}进行排序，若前三趟的排序结果如下：

第一趟：{2,12,16,5,10,88}

第二趟：{2,12,5,10,16,88}

第三趟：{2,5,10,12,16,88}

则采用的排序算法可能是（）。

A.冒泡排序 B.希尔排序

C.归并排序 D.基数排序